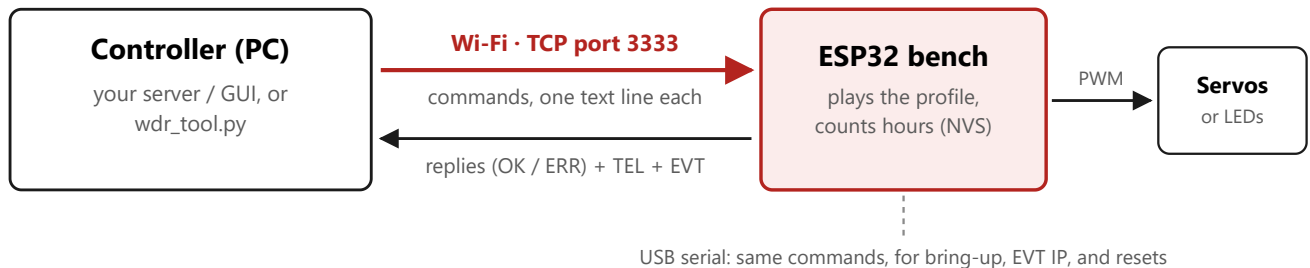


WDR: Welkinrim Durability Replay

User manual and protocol v1 for the flight-data replay bench. The bench replays a recorded flight into servos again and again, and counts every cycle and every working hour.

1 How the pieces fit



THE IDEA

A flight log is turned into a **profile**: a table of servo pulse widths (μ s), one row per **frame**. The bench plays the frames at a fixed rate. One pass is a **cycle**, and it loops for hours.

THE ONE RULE TO REMEMBER

Every command line gets **exactly one** reply line, starting with **OK** or **ERR**. Send a command, wait for its reply (up to 1 s), then send the next.

2 Quick start in 5 minutes, no hardware needed

1 Install Python 3, then run `pip install pyserial`. That is the tool's only dependency.

2 Start a simulated bench on your own PC. It behaves exactly like the real one:

```
python wdr_tool.py serve-sim          # listens on 127.0.0.1:3333, leave it running
```

3 In a second window, talk to it by hand:

```
python wdr_tool.py term --host 127.0.0.1
> PING
OK PONG
> INFO
OK INFO proto=1 team=SIM ch=4 maxframes=8000 up=12
```

4 Point your own controller at `127.0.0.1:3333`. Open a TCP socket, write lines ending in `\n`, and read lines back.

5 On the day, swap `127.0.0.1` for the real bench's IP. Nothing else changes.

3 The tool: `wdr_tool.py`

COMMAND	WHAT IT DOES
<code>serve-sim</code>	Runs a simulated bench on <code>127.0.0.1:3333</code>
<code>term --host <ip></code> or <code>term --serial COM7</code>	Type commands by hand and watch replies, TEL and EVT live
<code>makecsv flight.csv</code>	Writes a sample flight profile (4 channels, 50 Hz, 4 s)
<code>upload flight.csv --host <ip> --rate 50</code>	Sends a CSV to the bench and checks SUM
<code>test --host <ip> --serial COM7</code>	Conformance scorer for bench firmware, 7 levels, out of 85
<code>test --port sim</code>	The same scorer against the built-in simulator (a perfect score)
<code>soak --host <ip> --minutes 30</code>	Long endurance run that counts errors and timeouts

Only one program can hold a serial port. Close the Arduino Serial Monitor or `idf.py monitor` first. The board prints its IP on USB as `EVT IP 192.168.x.x` once it joins Wi-Fi.

4 The link

	WI-FI: THE CONTROL LINK	USB SERIAL: BRING-UP AND DEBUG
Settings	Station mode on the event network (2.4 GHz). TCP server on port 3333	115200 baud, 8N1
Framing	One command per line, ending in <code>\n</code> (<code>\r</code> is ignored). Commands are UPPERCASE, words separated by one space. Max 128 characters	
Special lines	An empty line is ignored and gets no reply. A line over 128 characters is thrown away and answered with ERR ARGS	
Other output	Lines that don't start with OK , ERR , TEL or EVT are debug text. Controllers must ignore them	

5 Wi-Fi rules

- 1 The bench joins the event network **by itself** after every boot, with no key press and no PC.
- 2 When it gets an address it prints `EVT IP <address>` on USB serial, and again whenever the address changes.
- 3 **One controller at a time.** When a new client connects, the old one is closed. A crashed PC can always reconnect.
- 4 **The bench never depends on the link.** If the client disappears, playback and counting carry on, on time. The bench accepts a new client by itself.
- 5 Replies go to whoever sent the command. `TEL` and `EVT` go to the TCP client and to USB.
- 6 `TEL` is switched **off** whenever a new client connects. Send `TEL 1` again after reconnecting.

6 Concepts

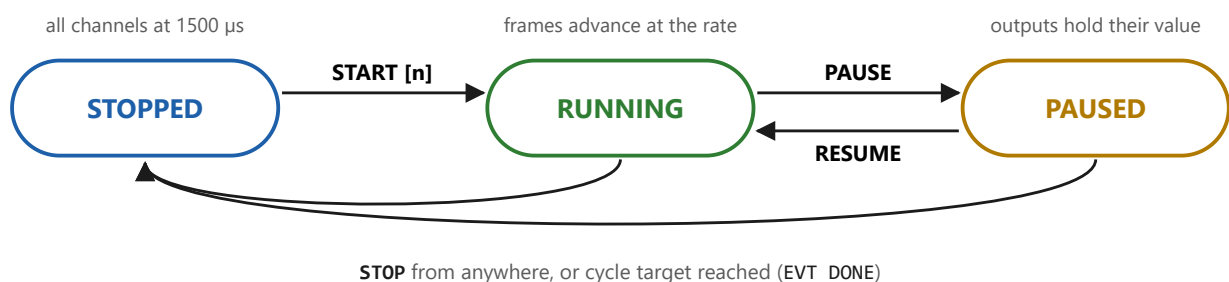
- **Channel:** one PWM output, numbered `0 ... N-1` (N is 1 to 8, usually 4). Servo PWM at **50 Hz**, pulses **500 to 2500 μ s**.
- **Idle:** 1500 μ s (servo centre). Every channel is at idle after boot and after `STOP`.
- **Profile:** frames $\times$ channels of μ s values, played at **10 to 100 frames/s**. At least 500 frames are always supported. `INFO` reports the real limit as `maxframes`.
- **Cycle:** one pass through all frames. A cycle lasts $\text{frames} \div \text{rate}$ seconds.

COUNTERS (LIFETIME, SURVIVE POWER LOSS)

- `cycles` : completed cycles, ever
- `run_s` : seconds spent RUNNING
- `active_s` , per channel: seconds RUNNING while the channel was **more than 25 μ s from 1500**

Saved at least once a minute while running, and on every `STOP`, `PAUSE` and `DONE`. So a power cut loses at most 60 s.

7 States



After a reset or power cut the bench always comes back **STOPPED** with counters intact, and never starts by itself. It is back on Wi-Fi within 20 s.

8 Command reference

`<x>` = required integer, `[x]` = optional. "Allowed in" shows which state the command is accepted in. In any other state the reply is `ERR STATE`.

COMMAND	ALLOWED IN	REPLY ON SUCCESS	NOTES
PING	any	OK PONG	Link check
INFO	any	OK INFO proto=1 team=<name> ch=<N> maxframes=<M> up=<s>	up = seconds since boot. If it goes down, the bench rebooted
SET <ch> <us>	STOPPED	OK	Drive one channel by hand. ch 0...N-1, us 500...2500
LOAD <rate> <frames>	STOPPED	OK	Starts an upload and discards the old profile. rate 10...100, frames 1...M
F <i> <us0> <us1> ...	after LOAD	OK	Frame i (0-based, in order) with exactly N values
COMMIT	all frames sent	OK SUM=<s>	s = sum of every μ s value, mod 65536. Check it against your own sum
START [cycles]	STOPPED	OK	Plays from frame 0. cycles = target, 0 or absent = forever
PAUSE	RUNNING	OK	Outputs hold. Counters are saved
RESUME	PAUSED	OK	Carries on from the same frame
STOP	any	OK	Every channel goes to 1500, frame to 0. Always allowed
STATUS	any	OK STATUS state=<S> cycle=<c> target=<t> frame=<f> frames=<n> us=<a,b,...>	cycle = cycles done this run. us = live output of each channel
COUNTERS	any	OK COUNTERS cycles=<c> run_s=<r> active_s=<a,b,...>	Lifetime totals, whole seconds
CLEAR	STOPPED	OK	Resets all counters to 0 and saves
TEL <0 1>	any	OK	Telemetry stream off or on. Off after boot and for every new client
TIME <unix>	any	OK	PC clock in seconds since 1970 (optional)

9 Errors

CODE	MEANING
UNKNOWN	Command word not recognised
ARGS	Wrong number of arguments, not a number, or line too long
RANGE	Number out of range (channel, μ s, rate, frames)
STATE	Not allowed in the current state
NOPROFILE	START without a committed profile
FULL	Profile larger than maxframes

Format: `ERR <CODE> [free text]`. Programs check only the code word. The text is for humans.

10 Uploading a profile

- LOAD, then F 0 ... F n-1 in order, then COMMIT. Wait for each OK.
- A wrong index or the wrong number of values gets `ERR ARGS`. A value outside 500...2500 gets `ERR RANGE`. Either way the upload is **aborted**: start again with LOAD.
- COMMIT before every frame has arrived gets `ERR STATE`.
- Compare SUM with your own sum mod 65536. If they differ, re-upload.

CSV FILE FORMAT

```
t_ms,ch0,ch1,ch2,ch3
0,1500,1500,1500,1500
20,1512,1490,1500,1530
```

11 Telemetry and events

TELEMETRY (AFTER TEL 1), EVERY 500 MS ± 100 MS

```
TEL state=RUNNING cycle=3 frame=120 us=1500,1620,1410,1500
```

EVENTS (ALWAYS ON)

LINE	WHEN
EVT BOOT	Once, when the firmware has started (on USB)
EVT IP <addr>	Wi-Fi got an address (on USB)
EVT CYCLE <n>	A cycle finished. n = cycles done this run
EVT DONE	The cycle target was reached. The bench is now STOPPED

TEL and EVT can arrive between a command and its reply. A controller should route lines by their first word, not assume the next line is the reply.

12 A full session

```
> PING
< OK PONG
> INFO
< OK INFO proto=1 team=RoboRiders ch=4 maxframes=8000 up=42
> LOAD 50 3
< OK
> F 0 1500 1500 1500 1500
< OK
> F 1 1600 1400 1500 1700
< OK
> F 2 1700 1300 1500 1900
< OK
> COMMIT
< OK SUM=18600 # 6000+6200+6400
> START 2
< OK
< EVT CYCLE 1
< EVT CYCLE 2
< EVT DONE
> STATUS
< OK STATUS state=STOPPED cycle=2 target=2
    frame=0 frames=3 us=1500,1500,1500,1500
> SET 9 1500
< ERR RANGE channel 9 does not exist
> FLY
< ERR UNKNOWN
```

Blue = controller sends, red = bench replies.

13 Things to think through

TIMING UNDER LOAD

Wi-Fi traffic, flash writes and printing all take time. How do you make sure a frame is **never late** while a client floods the bench with commands?

A VANISHED CLIENT

The laptop lid closes mid-run. What happens the next time something is sent to it? How is a new client noticed and served?

POWER CUT

Power goes 59 s after the last save. What is lost, and is that acceptable? What if the cut comes **during** a save?

SHARED STATE

The network code and the player both touch the same state. What stops either of them seeing it half-updated?

14 Troubleshooting

SYMPTOM	CHECK
Can't connect to port 3333	Did the bench print EVT IP ? Are the PC and bench on the same 2.4 GHz network? Many campus networks block device-to-device traffic, so use the event router
My connection closed by itself	Another client connected and took over (rule 3). Reconnect
Telemetry stopped after reconnecting	That is expected (rule 6). Send TEL 1 again
Replies seem one line out of step	Your reader treated a TEL or EVT line as a reply. Route lines by their first word
Serial port busy or access denied	Close the Arduino Serial Monitor or idf.py monitor